UDESC



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

UDESC

Traits

Rust suporta o polimorfismo por meio de duas características: generics e traits. Traits são inspiradas em classes de tipos de Haskell, mas também podem ser comparadas com interfaces ou classes abstratas de linguagens orientadas a objetos, por exemplo:

```
pub trait Display {
   fn fmt(&self, f: &mut Formatter<'_>) -> Result<(), Error>;
}
```

UDESC

Trait: Eq

`x.eq(y)`também pode ser escrito como `x == y` e `x.ne(y)` pode ser escrito como `x != y`.

```
pub trait PartialEq<Rhs = Self>
where
    Rhs: ?Sized,
{
    fn eq(&self, other: &Rhs) -> bool;
    fn ne(&self, other: &Rhs) -> bool { !self.eq(other) }
}
pub trait Eq: PartialEq { // Vazio  }
```

Rhs = Self (Parâmetro Genérico com Default): O parâmetro Rhs representa o tipo do lado direito (*Right-Hand Side*) da operação. Por padrão, ele assume o valor Self (o mesmo tipo que está implementando a trait). Permite que a implementação compare valores de tipos distintos



Trait: Display

```
#[derive(Debug)]
struct Ponto {
    x:i32, y:i32
}

fn main() {
    let p1 = Ponto{x:1, y:10};
    println!("{p1:?}");
 }
```

UDESC

Trait: Display

```
use std::fmt;
#[derive(Debug)]
struct Ponto {
    x:i32, y:i32
}

impl fmt::Display for Ponto {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({},{})", self.x, self.y)
    }
}


fn main() {
    let p1 = Ponto{x:1, y:10};
    println!("{p1:?}");
    println!("{p1}");
 }
```

UDESC

Trait: Display

```
use std::fmt;
enum Semana{
    Dom, Seg, Ter, Qua, Qui, Sex, Sab
}

use Semana::{*};

impl fmt::Display for Semana {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
    let dia = match *self {
        Dom => "Domingo",
        Seg => "Segunda",
        Ter => "Terça",
        Qua => "Quarta",
        Qui => "Quinta",
        Sex => "Sexta",
        Sab => "Sábado"
    };
    write!(f, "{dia}")
    }
}
```

UDESC

Trait: Display

```
impl<T:Display + Clone + Copy> fmt::Display for Lista<T> {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        let mut p = &self.cabeca;
        write!(f, "[")?;
        loop {
            match **p {
                Cons {cab: ref x, cauda:ref xs} => {
                    p = xs;
                    write!(f, "{x}")?;
                    if !xs.vazio() {write!(f, ", ")?;}
                }
                Nil => {

                    break write!(f, "]");
                }
            }
        }
    }
}
```

UDESC

Trait: PartialEq

```
use std::cmp::PartialEq;
enum Semana{
    Dom, Seg, Ter, Qua, Qui, Sex, Sab
}

use Semana::{*};

impl PartialEq for Semana{
    fn eq (&self, r:&Self) -> bool {
        match *self {
            Dom => match *r {Dom => true, _ => false},
            Seg => match *r {Seg => true, _ => false},
            Ter => match *r {Ter => true, _ => false},
            Qua => match *r {Qua => true, _ => false},
            Qui => match *r {Qui => true, _ => false},
            Sex => match *r {Sex => true, _ => false},
            Sab => match *r {Sab => true, _ => false}
        }
    }
}
```



Trait: PartialOrd

```
pub trait PartialOrd<Rhs = Self>: PartialEq<Rhs>
where
    Rhs: ?Sized,
{
    fn partial_cmp(&self, other: &Rhs) -> Option<Ordering>;

    fn lt(&self, other: &Rhs) -> bool {
        matches!(self.partial_cmp(other), Some(Ordering::Less))
    }
    fn le(&self, other: &Rhs) -> bool { ... }
    fn gt(&self, other: &Rhs) -> bool { ... }
    fn ge(&self, other: &Rhs) -> bool { ... }
}

pub enum Ordering {
    Less = -1,
    Equal = 0,
    Greater = 1,
}
```

UDESC

Trait: Ord

```
pub trait Ord: Eq + PartialOrd {
    fn cmp(&self, other: &Self) -> Ordering;

    fn max(self, other: Self) -> Self
       where Self: Sized, {
       if other > self { other } else { self }
    }
    fn min(self, other: Self) -> Self
       where Self: Sized, { ... }
    fn clamp(self, min: Self, max: Self) -> Self
       where Self: Sized, { ... }
}
```

max, **min** e **clamp** exigem que *Self Sized* porque recebem e retornam valores de tipo *Self* por valor, o que não é possível para tipos de tamanho desconhecido.

UDESC

Trait: Add

```
pub trait Add<Rhs = Self> {
        type Output;
        fn add(self, rhs: Rhs) -> Self::Output;
}
```

***type Output* (Tipo Associado)**: Define qual será o tipo retornado pela soma. Isso dá flexibilidade para que a adição de dois tipos resulte em um terceiro tipo completamente diferente.

UDESC

Trait: Add

```
use std::ops::Add;
#[derive(Debug)]
struct Ponto {
    x:i32, y:i32
}
impl Add<i32> for Ponto {
    type Output = Self;
    fn add(self, rhs: i32) -> Self::Output {
        Ponto {
            x: self.x + rhs,
            y: self.y + rhs,
        }
    }
}

fn main() {
    let mut p1 = Ponto{x:1, y:10};
    p1 = p1 + 10;
    println!("{p1}");
}
```